Product RAG API 연동 문서

기준일: 2026-10-19 (Asia/Seoul) · 문서 v1.0 · 실행 API 0.3.0

1. 연결 주소와 역할

Base URL: https://rag.chanuk.theworkpc.com

다른 PC에서는 HTTPS 도메인으로 호출한다. 현재 NPM(Nginx Proxy Manager)이 Spark의 172.30.1.53:8002로 전달하며, Tailscale·SSH·DB 비밀번호 없이 호출할 수 있다. API 자체의 별도 인증키 검증은 현재 구현되어 있지 않다. 인증 상태가 바뀌면 문서를 함께 갱신해야 한다.

항목	현재 값
상태 확인	GET /healthz
검색	POST /v1/rag/search
기본 프로필	test_items2
선택 가능한 프로필	test_items2, all_products
test_items2 카테고리	beauty, fashion, food, living
test_items2 원본	상품 1,500개 / 옵션 1,917개 / 리뷰 46,100개
test_items2 기존 색인	청크 114,106개, 384차원
기본 모델	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
DB	기존 PostgreSQL 5432: toss_rag / Commerce 조회 toss

이 API는 상품 후보·가격/재고 조회 결과·검색 근거를 반환한다. 최종 자연어 답변을 생성하는 LLM API가 아니며, 장바구니·주문·결제 작업도 하지 않는다. 5432는 DB 포트, 8002는 RAG HTTP API 포트다.

도메인 루트 /에는 웹 화면이 없어 **404**가 정상이다. 연결 확인에는 /healthz를 사용한다.

2. 가장 빠른 호출 방법

2.1 상태 확인

```
curl https://rag.chanuk.theworkpc.com/healthz
```

현재 확인한 응답:


```
{
  "status": "ok",
  "profile_name": "test_items2",
  "dataset_version": "test_items2:v6.1-review-refined:sha256=94681a28271ca9b1:products=1500:reviews=46100",
  "supported_categories": ["beauty", "fashion", "food", "living"],
  "allowed_profiles": ["test_items2", "all_products"],
  "commerce_configured": true
}
```

healthz는 HTTP 200이어도 본문 status가 error 또는 not_ready일 수 있다. HTTP 상태와 본문 status를 함께 확인한다. commerce_configured=true는 Commerce 객체가 구성되었다는 뜻이며, 모든 상품의 매핑·추천 품질이 검증됐다는 뜻은 아니다.

2.2 Postman

1. Method를 POST로 선택하고 https://rag.chanuk.theworkpc.com/v1/rag/search를 입력한다.
2. Body → raw → JSON을 선택한다. Content-Type은 application/json을 사용한다.
3. 아래 JSON을 넣고 Send를 누른다. 제공한 Postman collection을 Import해도 된다.

```
{
  "request_id": "other-pc-test-001",
  "profile": "test_items2",
  "original_question": "니플패치 찾아줘. 리뷰도 보여줘",
  "query": "니플패치",
  "category": "fashion",
  "filters": {},
  "required_conditions": [],
  "preferences": [],
  "top_k": 5
}
```

이 검색 예시는 사용법이다. 이 문서 작성 중 검색 POST를 실행하거나 니플패치의 검색 성공을 검증한 결과가 아니다.

2.3 curl — macOS/Linux

요청 예시 파일 examples/rag-search.request.json을 내려받은 뒤 실행한다.

```
curl --request POST \
  'https://rag.chanuk.theworkpc.com/v1/rag/search' \
  --header 'Content-Type: application/json' \
  --data-binary @rag-search.request.json
```

Windows PowerShell에서는 curl.exe를 명시하고 한 줄로 실행할 수 있다.

```
curl.exe -X POST "https://rag.chanuk.theworkpc.com/v1/rag/search" -H "Content-Type: application/json" --data-binary "
@rag-search.request.json"
```

3. 요청 필드

Content-Type: application/json · UTF-8 · 요청 본문 최대 **64 KiB**.

필드	타입 / 기본값	규칙
original_question	string, 필수	사용자 원문. 1~4,000자, 공백만 불가
query	string, 필수	검색에 쓸 핵심어/질문. 1~1,000자, 공백만 불가
request_id	string 또는 null / null	추적용 ID, 최대 128자. 자동 생성·중복 제거 키가 아님
profile	string 또는 null / 서버 기본값	현재 기본 test_items2. all_products도 허용
category	string 또는 null / 전체	프로필 내부 검색 범위. 1~128자
product_type	string 또는 null / 자동 추론	등록된 유형 ID 또는 지원되는 카탈로그 유형. 1~128자
filters	object / {}	아래 가격·재고 조건. null 불가

필드	타입 / 기본값	규칙
required_conditions	string[] / []	최대 20개, 각 1~500자. 임의 조건은 미검증으로 반환
preferences	string[] / []	최대 20개, 각 1~500자. 임의 선호는 미검증으로 반환
top_k	integer / 10	1~20. 실제 반환 수는 더 적을 수 있음

미등록 요청 필드는 422 invalid_request다. 원문과 query는 모두 필요하며, 원문에 있는 상품을 재작성 과정에서 다른 상품군으로 바꾸지 않는다. 숫자·문자열 타입을 임의로 바꾸거나 top_k에 문자열을 보내지 않는다.

자연어 문자열을 넣을 수 있지만 API가 범용 LLM 의도 분석이나 대화 기억을 수행하는 구조는 아니다. Brain에서 목적·카테고리를 판단하고 짧은 상품 핵심어를 query로 전달하는 방식을 권장한다. 예: original_question은 “운동 후 먹을 단백질 보충용 상품 추천”, query는 “단백질 보충식”.

3.1 프로필과 카테고리

profile	실제 데이터 범위	지원 category
test_items2	새 통합 데이터 1,500상품	beauty, fashion, food, living
all_products	이전 test_items 180상품	beauty

프로필 이름 all_products가 전체 최신 상품을 뜻하지 않는다. 현재는 이전 뷰티 색인이다. 새 데이터는 test_items2를 명시하거나 기본값을 사용한다. category 생략 시 선택한 프로필의 전체 카테고리에서 검색한다. 니플패치를 beauty로 제한하면 fashion 상품은 제외된다. 프로필에 없는 category는 unsupported_category다.

3.2 가격·재고 필터

filters 내부 필드	타입 / 기본값	적용
price_min	number 또는 null / null	0 이상, 옵션 단가 하한 포함
price_max	number 또는 null / null	0 이상, 옵션 단가 상한 포함
currency	string 또는 null / null	가격 범위 지정 시 필수. 현재 KRW만 지원
in_stock	boolean 또는 null / 서버 기본값	현재 생략/null은 true. false는 재고 0인 옵션도 허용

```
{
  "price_min": 10000,
  "price_max": 20000,
  "currency": "KRW",
  "in_stock": true
}
```

price_min ≤ price_max여야 한다. 가격과 재고는 **같은 옵션**에서 확인하며 조건에 맞는 최저 단가 옵션을 고른다. 배송비·수량·쿠폰·총 결제액 필터는 구현되어 있지 않다. 예산 총액을 옵션 단가와 동일하게 취급하지 않는다.

required_conditions에 “통기성 좋음”을 넣어도 해당 조건을 만족하는 상품만 보장해서 거르는 것은 아니다. 현재는 unverified_conditions로 반환하며 별도 근거 확인이 필요하다.

product_type은 합의한 유형만 보낸다. 예: detergent, protein_food, pouch. 니플패치는 현재 별도 등록 유형이 없으므로 예시처럼 query를 보내고 product_type은 생략한다. 이 필드에 임의 문장을 넣는 것은 범용 분류 요청이 아니다.

4. 응답 구조와 상태

검색 응답에서 먼저 HTTP 상태를 확인하고, 이어 본문 status와 error_code를 처리한다.

본문 status	일반 HTTP	처리
ok	200	items 사용. 조건 검증·출처·Commerce 상태도 함께 확인
no_results	200	적격 후보 없음. search_context.reason 확인
low_relevance	200	유형 충돌 또는 카탈로그 정체성 일치 부족. 확인 질문 등으로 처리
error	422/500/503/504	error_code에 따라 입력 수정 또는 서버 확인

4.1 검색 응답 예시

아래는 **형식 설명용 가상 값**이며 실제 검색 실행 결과가 아니다. 전체 예시는 examples/rag-search.response.example.json에 있다.


```
{
  "request_id": "other-pc-test-001",
  "status": "ok",
  "profile_name": "test_items2",
  "dataset_version": "<현재 색인 버전>",
  "applied_filters": {"category": "fashion", "in_stock": true},
  "supported_categories": ["beauty", "fashion", "food", "living"],
  "unverified_conditions": [],
  "items": [{
    "product_id": "example-product-001",
    "category": "fashion",
    "product_metadata": {"title": "설명용 상품명"},
    "rrf_score": 0.03,
    "evidence": [{"source_type": "review", "source_id": "example-review-001", "evidence_text": "설명용 인용문입니다."}],
    "commerce": {"variant_id": "example-variant-001", "validation_status": "verified_database_row"},
    "ranking": {"relevance_qualified": true, "title_identity_match": 1, "lexical_coverage": 1.0, "requested_product_type": null}
  }],
  "error_code": null,
  "search_context": {"ranking_version": "product-hybrid-v2", "relevance_gate": "product_identity_v2"}
}
```


위 예시는 필드를 일부 생략한 발췌다. 실제 요청 결과의 근거 출처와 데이터 버전을 확인하고, 실제 인용에는 반환된 evidence_text를 사용한다.

4.2 공통 필드

필드	의미
request_id	요청 추적값. 오류 처리 과정에서 일부 문자가 정리될 수 있음
profile_name	실제 선택한 프로필
dataset_version	실제 활성 색인 버전. 실패 단계에 따라 null
applied_filters	적용한 카테고리·유형·가격·재고 조건
supported_categories	선택한 프로필의 카테고리 목록
unverified_conditions	적용을 보장하지 못하는 필수/선호 조건
items	상품 단위 결과 배열
error_code	성공/빈 결과이면 null, 오류이면 코드
search_context	검색 버전·유형 추론·후보 수·빈 결과 이유 등 진단 정보

입력 검증 실패나 예외로 반환하는 오류 응답은 profile_name/search_context가 없을 수 있다. 클라이언트는 이 필드의 항상 존재를 가정하지 않는다. search_context의 세부 키는 처리 단계에 따라 달라진다.

5. 상품·옵션·리뷰 근거의 의미

결과 필드	의미와 사용법
items[].product_id	Commerce 원본 상품과 연결할 ID. 프로필/출처와 함께 사용
category	반환 상품의 실제 대분류
product_metadata	title, brand, main_category, price, average_rating, rating_number, source_dataset, dataset_release_status 중 저장된 필드
commerce	DB에서 조회한 선택 옵션·단가·재고·시각·출처 상태
rrf_score	검색 순위 결합 점수. 정답 확률·효능·만족도 점수 아님
ranking.title_identity_match	전체 정규화 제목 일치 2 / 제목 토큰 일치 1 / 제목 토큰 일치 없음 0
ranking.lexical_coverage	제목·브랜드·유형에 대한 어휘 일치 진단값
ranking.relevance_qualified	현재 구현의 관련성 검사를 통과했다는 값. 사용자 조건 전체 충족 보증 아님
evidence	상품당 최대 3개의 선택 근거 청크

검색 응답 product_metadata에는 이미지 URL·전체 옵션 목록이 포함되지 않는다. 카드 이미지·상세 정보는 product_id와 source_dataset을 이용해 해당 Commerce 데이터에서 가져온다. 전체 평점과 리뷰 수는 저장된 경우에만 포함되며, 검색된 일부 리뷰의 평균으로 전체 평점을 덮어쓰지 않는다.

5.1 commerce

commerce에는 variant_id, unit_price_won, currency, stock_qty, checked_at, validation_status, source_database, source_schema, price_basis, is_simulated, purchase_mode, is_live_market_price, dataset_release_status, source_price_evidence_status가 들어갈 수 있다.

현재 test_items2는 purchase_mode=demo, is_live_market_price=false다. verified_database_row는 DB 행과 가격·재고를 확인했다는 뜻이며 실제 시장 재고·실시간 판매 가격을 보장하지 않는다. is_simulated는 원본 표식을 전달하므로 boolean 또는 null일 수 있다. stock_qty와 unit_price_won도 시연 데이터라는 상태를 유지해서 표시한다.

5.2 evidence

근거 필드	의미
product_id / chunk_id	상품 ID / 청크 ID. 청크 ID는 리뷰 ID가 아님
source_type / source_id	review이면 원본 리뷰 출처 ID. 상품 근거이면 상품 출처 ID
source_field	색인에 사용한 정규화 문서의 필드 경로. 예: /text
source_version	해당 활성 데이터셋 버전
evidence_text	응답에서 실제 인용할 수 있는 근거 문자열
source_metadata	source_kind, synthetic, variant_id, rating, source_table, source_dataset, source_ref 등. 일부 키는 없거나 null
start_offset / end_offset	정규화 원문 필드 내 0부터 시작하는 문자 구간 [start, end)
bm25_rank / vector_rank	해당 검색 채널 내 청크 순위. 미포함 시 null
review_bm25_rank / review_vector_rank	리뷰 의도 시 추가되는 리뷰 전용 채널의 순위. null 가능
rrf_rank	반환 상품의 순서. 리뷰 평점이나 근거 신뢰도 점수 아님

review의 source_kind=REAL/SYNTHETIC을 보존하고 합성 리뷰를 실제 구매 경험으로 표시하지 않는다. source_metadata.variant_id=null인 상품 단위 리뷰도 허용한다. 이를 선택된 commerce.variant_id에 대한 특정 옵션 리뷰로 바꾸어 표현하지 않는다.

5.3 ID 연결 계약

- test_items2 결과의 product_id는 toss.test_items2.products.id와 연결한다.
- test_items2의 review 근거 source_id는 toss.test_items2.reviews.id와 연결한다. source_table/source_dataset도 확인한다.
- commerce.variant_id는 선택된 원본 product_variants.id다. 리뷰 근거의 variant_id와 의미가 다르다.
- 다른 프로필의 같은 제목 상품은 다른 ID를 가질 수 있다. 제목으로 프로필 간 ID를 대체하지 않는다.
- source_field와 offset은 정규화 문서 기준이다. PostgreSQL body 문자열에 무조건 직접 적용하지 말고 전달된 evidence_text를 사용한다.
- 기존 /v1/catalog/test-items/{product_id}/reviews는 별도 legacy 응답으로 ID를 해시 변환한다. test_items2 리뷰 API로 사용하지 않는다.

6. 오류와 빈 결과 진단

6.1 검색 API 오류 코드

error_code	HTTP	의미 / 대응
invalid_request	422	필수 필드·타입·길이·미등록 필드·가격 범위·64 KiB 제한 확인
unsupported_profile	422	test_items2 또는 all_products 사용
unsupported_category	422	선택 프로필의 supported_categories 확인
unsupported_currency	422	현재 가격 필터는 KRW만 사용
index_not_ready	503	해당 프로필 활성 색인 확인
configuration_error	503	실행 모델과 색인의 모델 등 서버 설정 확인
commerce_not_configured	503	Commerce 연결 구성 필요
commerce_source_unmapped	503	원본 데이터셋과 Commerce 스키마 매핑 확인
embedding_failed	503	쿼리 임베딩 생성 실패. 서버 로그 확인
database_failed	503	DB 연결·권한·SQL 실행 상태 확인
timeout	504	API 내부 TimeoutError 처리. 프록시 자체 504와 구분
search_failed	500	검색 처리 예외. request_id와 서버 로그 대조


```
{
  "request_id": "other-pc-test-001",
  "status": "error",
  "dataset_version": null,
  "applied_filters": {},
  "supported_categories": [],
  "unverified_conditions": [],
  "items": [],
  "error_code": "invalid_request"
}
```


위 오류 JSON도 형식 예시다. 프록시 자체 오류는 JSON 대신 HTML을 반환할 수 있다. HTTP 오류 여부를 먼저 확인하고 JSON Content-Type/파싱 실패도 처리한다. 422는 같은 요청을 그대로 반복하지 말고 입력을 수정한다.

6.2 HTTP 200이지만 검색 결과가 없는 경우

search_context.reason	의미
conflicting_or_ambiguous_product_type	원문·재작성 검색어·명시 유형 충돌 또는 모호함
category_type_mismatch	지정 카테고리와 추론 유형의 범위 불일치
no_matching_product_type	해당 유형의 후보 없음
no_catalog_identity_match	제목·브랜드·유형의 어휘 연결 부족
no_commerce_eligible_products	가격·재고 등 Commerce 조건 충족 후보 없음
no_eligible_candidates	검색·관련성·근거 선택 후 최종 후보 없음

6.3 접속 문제

증상	확인할 내용
도메인 / 가 404	웹 화면이 없는 정상 상태. /healthz 사용
GET /v1/rag/search 가 405	검색은 POST. 브라우저 주소창 입력은 GET임
도메인에서 502	NPM이 upstream에 연결할 수 있는지 확인. 목적지는 http://172.30.1.53:8002
내부 health는 정상, 도메인은 502	LAN 수신/네트워크/NPM 경로 확인. 과거에는 127.0.0.1 전용 바인딩이 원인이었음
브라우저 JS에서만 실패	FastAPI 앱에 별도 CORS 허용 미들웨어가 없음. 현재 NPM의 CORS 설정은 미검증. 서버 측 호출 또는 같은 출처 프록시 사용
상품 카드는 있지만 AI 답변 실패	이 API의 상품 응답과 별도 Brain/LLM 설명 생성을 각각 확인
잘못된 상품이 노출	profile·category·query·유형·관련성 검사와 Brain의 후속 재정렬 확인

정상 통신이 검색 품질을 보장하지 않는다. 이미 존재하는 상품도 잘못된 프로필/카테고리·상품 유형·후속 어댑터에서 누락될 수 있다.

7. Node.js와 Python 호출 예시

예시는 서버 측 실행용이며 자동 테스트 스크립트가 아니다. 타임아웃 30초는 호출 예시의 선택값으로, 운영 SLA나 API 강제 제한을 뜻하지 않는다.

7.1 Node.js — 내장 fetch를 제공하는 실행 환경

```
const response = await fetch(
 'https://rag.chanuk.theworkpc.com/v1/rag/search',
 {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  signal: AbortSignal.timeout(30_000),
  body: JSON.stringify({
   request_id: 'node-example-001',
   profile: 'test_items2',
   original_question: '니플패치 찾아줘. 리뷰도 보여줘',
   query: '니플패치',
   category: 'fashion',
   top_k: 5
  })
 }
);
const text = await response.text();
let data;
try { data = JSON.parse(text); }
catch { throw new Error(`RAG proxy/non-JSON response: ${response.status}`); }
if (!response.ok || data.status === 'error') {
 throw new Error(`RAG ${response.status}: ${data.error_code ?? 'unknown'}`);
}
if (data.status !== 'ok') {
 console.log('검색 결과 없음:', data.search_context?.reason);
} else {
 console.log(data.items);
}
```

예를 들어 위 코드를 rag-search.mjs에 저장해 node rag-search.mjs로 실행할 수 있다. fetch의 네트워크 오류와 TimeoutError도 상위 호출부에서 처리한다.

7.2 Python — 표준 라이브러리

```
import json
import urllib.request
import urllib.error

payload = {
  'request_id': 'python-example-001',
  'profile': 'test_items2',
  'original_question': '니플패치 찾아줘. 리뷰도 보여줘',
  'query': '니플패치',
  'category': 'fashion',
  'top_k': 5,
}
request = urllib.request.Request(
  'https://rag.chanuk.theworkpc.com/v1/rag/search',
  data=json.dumps(payload, ensure_ascii=False).encode('utf-8'),
  headers={'Content-Type': 'application/json'},
  method='POST',
)
try:
  with urllib.request.urlopen(request, timeout=30) as response:
    data = json.load(response)
  if data['status'] == 'error':
    print('RAG 오류:', data.get('error_code'))
  elif data['status'] != 'ok':
    print('검색 결과 없음:', data.get('search_context', {}).get('reason'))
  else:
    print(json.dumps(data, ensure_ascii=False, indent=2))
except urllib.error.HTTPError as error:
  print('HTTP 오류:', error.code, error.read().decode('utf-8', errors='replace'))
except (urllib.error.URLError, TimeoutError, json.JSONDecodeError) as error:
  print('연결·시간 초과·응답 형식 오류:', error)
```

8. Shopping Brain 연동 기준

1. 사용자 원문은 original_question에 보존하고, 핵심 상품을 query로 만든다. 원문에 없는 가격·유형·효능 조건을 만들지 않는다.
2. 최신 전체 데이터는 profile=test_items2를 사용한다. 카테고리를 확정하기 어렵다면 생략한다. beauty 기본값을 모든 요청에 강제하지 않는다.
3. 이 API를 사용하도록 Brain의 실제 local/remote 검색 경로를 확인한다. 환경변수 이름은 해당 Brain 구현의 계약을 따른다.
4. 반환된 product_id와 출처로 Commerce 원본을 연결한다. ranking·Commerce·근거 출처를 유지하고 무관한 상품을 가격/리뷰 점수로 다시 올리지 않는다.
5. REAL과 SYNTHETIC, 상품 근거와 리뷰 근거, 상품 단위 리뷰와 옵션 단위 리뷰를 구분한다. REAL이나 variant_id=null인 리뷰를 일괄 거부하지 않는다.
6. unverified_conditions의 조건을 "검증 완료"로 표현하지 않는다. 근거가 없는 통기성·효능·성별 적합성도 추정하지 않는다.
7. 최종 LLM 설명 실패는 검색 실패와 구분한다. 후보가 있으면 "상품 후보는 찾았지만 추천 이유와 리뷰 설명을 생성하지 못했어요"처럼 상태를 정확히 표시한다.

9. 현재 구현 범위와 제한

현재 검색은 상품 정체성 검사 → Commerce 가격/재고 확인 → BM25+벡터 후보 회수 → 상품 단위 RRF → 관련성 재검사 → 근거 선택 순서다. 원문 또는 query에 리뷰/후기/사용평 등이 있으면 리뷰 전용 검색 채널을 추가한다. 벡터 유사도만으로 상품군을 확정하지 않는다.

- 상품 유형 추론은 등록된 분류/별칭 기반이며 모든 상품군·복합명·오탈자·사용 목적을 이해하는 범용 분류기가 아니다.
- 전체 정규화 제목 일치 경로와 어휘 일치 우선순위는 있으나, 핵심 상품명·모델번호의 독립 후보 회수 및 최종 Brain 순위 보존 개선 요청이 모두 구현된 것은 아니다.
- required_conditions/preferences의 임의 조건은 현재 실제 필터·개인화 점수로 검증 적용되지 않는다.
- 검색된 근거는 원본 리뷰 전체가 아니며, 해당 상품의 모든 리뷰가 응답에 실리는 구조도 아니다.
- API 응답 순위와 외부 화면의 최종 순위는 Brain의 처리에 따라 달라질 수 있다.
- 기존 색인은 재사용했다. 배포된 청커 v3 코드가 기존 114,106개 청크를 v3로 바꿨다는 뜻은 아니다.
- 검색 요청만으로 카탈로그에 없는 상품을 생성하거나 구매 가능 상태를 만들지 않는다.

10. 기존 목록 API 및 자동 문서

경로	현재 범위
GET /v1/catalog/test-items?limit=50&cursor=0	이전 all_products/test_items 뷰티 목록에 고정
GET /v1/catalog/test-items/{product_id}/reviews	이전 test_items 리뷰 복원 응답. 리뷰 ID를 별도 해시로 반환
GET /docs	FastAPI 자동 문서
GET /openapi.json	실행 API 경로 정보

기존 목록의 limit은 1~50, cursor는 숫자 offset 0~10,000이다. 응답은 total, items, next_cursor, has_more 등을 포함한다. 이 경로에 profile=test_items2를 붙여도 새 데이터 목록으로 전환되지 않는다. test_items2 전용 전체 목록·리뷰 조회 API는 현재 제공하지 않는다.

검색 요청은 API 내부에서 JSON을 직접 검증하므로 자동 OpenAPI 문서에 검색 Body 모델이 충분히 나타나지 않을 수 있다. 제공한 request.schema.json과 이 문서의 추가 검증 규칙, Postman collection을 함께 사용한다. JSON Schema만으로 공백·배열 요소 길이·가격 상하한 관계 같은 사용자 정의 검증이 모두 표현되지는 않는다.

11. 검증 범위·버전·전달 파일

- 실행 소스 커밋: 24a1051fff84bb4cf5441e90824e9e9be20e18ec.
- API 버전 0.3.0, ranking_version=product-hybrid-v2, relevance_gate=product_identity_v2.
- 기본 활성 run: e925490f-4f34-4c03-9248-eb9ebdfcde48.
- 작성 중 공개 HTTPS /healthz에서 HTTP 200, status=ok, test_items2, 네 카테고리를 다시 확인했다.
- 요청 JSON Schema는 실행 컨테이너의 RagSearchRequest에서 내보냈다. 필드·상태·오류·제한은 현재 코드와 대조했다.
- 이번 문서 작성 중 검색 POST·추천 품질 평가·외부 Brain E2E를 실행하지 않았다. 응답의 가상 값·예시 코드를 실제 실행 결과로 해석하지 않는다.
- 문서·PDF·호출 예시는 로컬 저장 상태다. 이번 문서 작업의 새 커밋·푸시·main 병합·제품 코드 수정은 수행하지 않았다.

전달 파일:

- rag-api-guide-2026-10-19.md: 이 정본 문서.
- ../reports/rag-api-guide-2026-10-19.pdf: 공유용 PDF 요약 및 필드 참조.
- rag-api.postman_collection.json: Postman Import용 요청 3개. 자동 테스트 스크립트 없음.
- rag-search.request.schema.json: 실행 요청 모델의 JSON Schema.
- examples/rag-search.request.json: UTF-8 요청 파일.
- examples/rag-search.response.example.json: 상세 응답 형식 예시, 실제 검색 결과 아님.

소스 참조: Backend의 src/toss_ads/rag/api.py, schemas.py, retriever.py, commerce.py, intent.py 및 docs/rag-test-items2-api-activation-2026-10-19.md, docs/rag-npm-lan-2026-10-19.md.